

LAB 01 · CLOUD SECURITY · FULL GUIDE

Cost Visibility Dashboard

Build the system that catches a cloud bill going wrong — while there's still time to stop it.

FOR

Kyler Nats

GENERATED

20 September 2026

STEPS

24 across 14 phases

SCREENSHOTS

21 capture points

Contents

P0 Baseline

- 0.0 How to read these tasks
- 0.1 Capture the empty state

P1 Foundation

- 1.1 Resource group
- 1.2 Log Analytics workspace
- 1.3 Checkpoint

P2 Identity and access

- 1.3 Managed identity
- 1.4 Role assignment — least privilege

P3 Storage

- 1.5 Storage account — the security baseline
- 1.6 Storage container
- 1.7 Diagnostic settings

P4 Alerts

- 2.1 Action group
- 2.2 Cost anomaly alert

P5 The budget

- 3.1 Subscription budget

P6 Export and outputs

- 4.1 Daily cost export
- 5.1 Outputs
- 5.3 Final checkpoint

P7 Deploy

- 7.1 Plan
- 7.2 Apply

P8 Verify every control

- 8.1 Portal verification sweep

P9 Prove the control works

- 9.1 Test that least privilege denies

P10 Prove the alert reaches a human

- 10.1 Test the action group

P11 The dashboard

- 11.1 Add the cost tiles

P12 Next day — real data

- 12.1 Confirm the export landed

P13 Tear it down

- 13.1 Destroy and record the total

Every command in this guide is run from `projects/01-cost-visibility/` unless the command says otherwise. Terraform files live in `terraform/` inside that folder. Check progress any time with `./scripts/check.sh` — it never touches Azure and never costs anything.

P0

Baseline

Evidence of the 'before'. These can never be retaken once resources exist.

0.0 How to read these tasks

Every Terraform task in this lab is the same shape. Learn it once here and the other eleven are the same job with different words.

WHY IT MATTERS

The point of this page is that you stop needing step-by-step direction. Once the pattern is in your head, a task like "create a storage account with versioning" is enough.

PART	WHAT IT MEANS
<code>resource</code>	The keyword. You are declaring a thing that should exist.
<code>"azurerm_resource_group"</code>	The type . Comes from the provider. Never invented — always looked up.
<code>"main"</code>	The local name . Yours to pick. Only used to reference it elsewhere in your own code.
<code>{ ... }</code>	The arguments . Which ones exist, and which are required, is in the docs for that type.

Where everything lives. All Terraform for this lab goes in `projects/01-cost-visibility/terraform/`. One file per concern: `main.tf` (foundation, identity, storage), `monitoring.tf` (alerts), `budget.tf`, `export.tf`, `outputs.tf`. Terraform reads every `.tf` file in the folder and treats them as one config, so the split is purely for humans. Write each task where its TODO marker already sits.

HOW DO I FIND THE ARGUMENTS?

Go to <https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs> the azurerm provider docs. Use the search box on the left. Type the resource type, for example `resource_group`. Open it. Two sections matter: **Example Usage**, which is copyable shape, and **Argument Reference**, which tells you what is Required versus Optional. Required arguments are the only ones you strictly need; the rest are choices you are making on purpose.

HOW DO I POINT ONE RESOURCE AT ANOTHER?

Use `type.local_name.attribute`. So the resource group you declare as `resource "azurerm_resource_group" "main"` is referenced elsewhere as `azurerm_resource_group.main.name` or `.location` or `.id`.

This matters for more than typing. Referencing is how Terraform learns the order to build things in. If you hardcode the string `"rg-costvis"` instead, Terraform has no idea the two are related and may try to build them in the wrong order.

Which attributes exist is listed under **Attributes Reference** on each docs page.

WHAT IS THE LOOP?

Write a block, then run `./scripts/check.sh`. It shows remaining TODOs, runs `terraform fmt`, and runs `terraform validate`. Nothing touches Azure and nothing costs money, so run it as often as you like.

Validate catches wrong argument names, missing required arguments, and type errors. It does not catch a value that is legal but wrong, which is what the requirement checks at the end are for.

Expect validation to fail until the last task is done, because `workbook.tf` references resources you have not written yet. That is normal.

WHAT IF I GET STUCK?

In order: read the error, since Terraform names the file, line, and usually the exact argument. Then check the docs page for that resource. Then open the Level 1 hint on the task. Then use the tutor button on the step, which can read your actual file and tell you what is wrong.

Asking the tutor is not cheating. Opening Level 3 and copying the code without understanding it is the only thing that defeats the purpose.

0.1 Capture the empty state

Three screenshots proving nothing was pre-built, plus a machine-readable snapshot of the Azure API.

WHY IT MATTERS

Any claim you make later rests on this. A dashboard full of data means nothing if you can't show what it looked like before.

SNAPSHOT THE API

```
./scripts/snapshot.sh baseline
```

Already done. These three are captured and redacted. Tick this and move on.

SCREENSHOT

baseline-empty-rg — Portal → Resource groups. Should show only `NetworkWatcherRG`.

```
./scripts/capture.sh baseline-empty-rg "Resource groups before the lab"
```

SCREENSHOT

baseline-credit — Portal → Subscriptions → your subscription. Shows Owner role and the \$200 credit.

```
./scripts/capture.sh baseline-credit "Owner role and $200 credit before deployment"
```

SCREENSHOT

baseline-cost-analysis — Cost Management → Cost analysis. Must show `BUDGET: NONE` and `FORECAST UNAVAILABLE`.

```
./scripts/capture.sh baseline-cost-analysis "Cost analysis with no budget and no forecast"
```

Foundation

Open `terraform/main.tf` and work top to bottom. Terraform in one sentence: you describe what you want, and it makes Azure match.

1.1 Resource group

WRITE IT IN `terraform/main.tf` REPLACING `# TODO 1.1`

A folder. Everything in this lab goes inside it, and deleting the folder deletes everything in it.

REAL LIFE

This is exactly why cloud bills spiral. People create things in random places, forget them, and pay for years. One folder per project means one command cleans it all up.

WHY IT MATTERS

Tags are how you answer 'what did this project cost?'. Untagged resources are the reason companies can't attribute their own spending.

WHAT TO SET ON `AZURERM_RESOURCE_GROUP`

- `name` — `"rg-${var.name_prefix}"`, which gives `rg-costvis`
- `location` — `var.location`
- `tags` — `var.tags`

HINT

Search `azurerm_resource_group` at [the registry](#) and open Example Usage. It needs three arguments and nothing more.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_resource_group" "main" {  
  name      = "rg-${var.name_prefix}"  
  location  = var.location  
  tags      = var.tags  
}
```

1.2 Log Analytics workspace

WRITE IT IN `terraform/main.tf` REPLACING `# TODO 1.2`

Where logs go so you can search them later. Without it, logs are written and thrown away.

REAL LIFE

This is what you'd search after an incident to answer 'who accessed that, and when?'. Here it records who read your cost data.

WHY IT MATTERS

Writing `azurerm_resource_group.main.name` instead of the literal string tells Terraform this depends on that, so it builds them in the right order automatically.

WHAT TO SET ON `AZURERM_LOG_ANALYTICS_WORKSPACE`

- `name` — `"law-${var.name_prefix}"`
- `location` and `resource_group_name` — reference the group from 1.1 rather than retyping it
- `sku` — `"PerGB2018"` (first 5 GB/month ingested is free)
- `retention_in_days` — `var.log_retention_days`
- `tags`

Answer this in DECISIONS.md: retention defaults to 30 days, not the 730-day maximum. Why deliberately keep /less?

HINT

Search `azurerm_log_analytics_workspace`. The SKU you want is the pay-as-you-go one, and retention is measured in days.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_log_analytics_workspace" "main" {
  name                = "law-${var.name_prefix}"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  sku                = "PerGB2018"
  retention_in_days = var.log_retention_days

  tags = var.tags
}
```

1.c Checkpoint

Confirm the loop works before going further.

RUN THE CHECKER

```
cd ~/Portfolio/projects/01-cost-visibility && ./scripts/check.sh
```

Expect **10 remaining** and `Success! The configuration is valid.` Send me the output before continuing — if your structure needs fixing, better to catch it now than at task 12.

Identity and access

The security core of the lab.

1.3 Managed identity

WRITE IT IN `terraform/main.tf` REPLACING `# TODO 1.3`

A login for software instead of a person. No password exists, so no password can leak.

REAL LIFE

The classic breach: a developer hardcodes a password, pushes it to GitHub, and a bot finds it within minutes. A managed identity removes the password entirely — Azure handles it invisibly.

WHAT TO SET ON `AZURERM_USER_ASSIGNED_IDENTITY`

- `name` — `"id-${var.name_prefix}-reader"`
- `location`, `resource_group_name`, `tags`

HINT

Search `azurerm_user_assigned_identity`. It takes the same four basics as the resource group plus a location.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_user_assigned_identity" "cost_reader" {
  name          = "id-${var.name_prefix}-reader"
  location      = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags          = var.tags
}
```

1.4 Role assignment — least privilege

WRITE IT IN `terraform/main.tf` REPLACING `# TODO 1.4`

The identity above can currently do nothing at all. This grants it exactly one permission: read cost data.

REAL LIFE

Ransomware spreads because service accounts have far more access than they need. One over-permissioned backup account can end a company. Least privilege asks: if this is stolen, what's the worst that happens? Here the answer is 'someone learns what your bill was.'

WHY IT MATTERS

A managed identity has two IDs. `principal_id` is *who it is*, used for permissions. `client_id` is *how apps address it*. Using the wrong one produces an error that doesn't explain itself.

WHAT TO SET ON `AZURERM_ROLE_ASSIGNMENT`

- `scope` — `data.azure_rm_subscription.current.id`. Billing lives at subscription level, not inside your resource group
- `role_definition_name` — "Cost Management Reader"
- `principal_id` — the identity's `principal_id`

HINT

Search `azurerm_role_assignment`. Three arguments: where the permission applies, which permission, and who gets it. Think about which of the identity's two IDs represents *who it is*.

REFERENCE SOLUTION — TRY IT YOURSELF FIRST

```
resource "azurerm_role_assignment" "cost_reader" {
  scope                = data.azure_rm_subscription.current.id
  role_definition_name = "Cost Management Reader"
  principal_id         = azurerm_user_assigned_identity.cost_reader.principal_id
}
```

P3

Storage

'Misconfigured storage bucket' is one of the most common breach headlines there is. This phase is about not being that.

1.5 Storage account — the security baseline

WRITE IT IN `terraform/main.tf` REPLACING `# TODO 1.5`

Cloud file storage. Your daily bill copies land here.

REAL LIFE

Millions of records get exposed this way every year, always the same root cause: someone left a container public. Every setting below is a control that has prevented a real breach somewhere.

WHY IT MATTERS

Versioning plus soft delete is ransomware defense. Ransomware encrypts files by overwriting them. With versioning on, the originals are still there.

WHAT TO SET ON `AZURERM_STORAGE_ACCOUNT`

- `name` — `"st${var.name_prefix}${random_string.suffix.result}"`. No hyphens allowed, and it must be globally unique across all of Azure
- `account_tier` `"Standard"`, `account_replication_type` `"LRS"`, `account_kind` `"StorageV2"`
- **HTTPS-only traffic** — no unencrypted connections
- **Minimum TLS 1.2** — blocks old, breakable encryption
- **Public access disabled** — this is the headline one
- A `blob_properties` block with versioning on and 7-day soft delete for both blobs and containers

HINT

Search `azurerm_storage_account`. The security arguments are near the top of the docs page; data protection lives inside a nested `blob_properties` block. One argument name changed in provider v4 — if you use the v3 name you'll get a clear error, so read it.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_storage_account" "exports" {
  name                = "st${var.name_prefix}${random_string.suffix.result}"
  location             = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  account_tier          = "Standard"
  account_replication_type = "LRS"
  account_kind           = "StorageV2"

  https_traffic_only_enabled = true
  min_tls_version            = "TLS1_2"
  allow_nested_items_to_be_public = false

  blob_properties {
    versioning_enabled = true

    delete_retention_policy {
      days = 7
    }

    container_delete_retention_policy {
      days = 7
    }
  }

  tags = var.tags
}
```

1.6 Storage container

WRITE IT IN `terraform/main.tf` REPLACING `# TODO 1.6`

A folder inside the storage account.

REAL LIFE

`private` means nobody on the internet can read it without credentials. Older systems defaulted to public, which is precisely how those breaches happened.

WHAT TO SET ON `AZURERM_STORAGE_CONTAINER`

- `name` — `"cost-exports"`
- `storage_account_id` — reference the account from 1.5
- `container_access_type` — `"private"`

HINT

Search `azurerm_storage_container`. In provider v4 it takes the account by ID, not by name — that changed from v3.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_storage_container" "exports" {
  name                = "cost-exports"
  storage_account_id  = azurerm_storage_account.exports.id
  container_access_type = "private"
}
```

1.7 Diagnostic settings

WRITE IT IN `terraform/main.tf` REPLACING `# TODO 1.7`

Turns on logging for the storage account and sends it to the workspace from 1.2.

REAL LIFE

After any breach the first question is 'what did they access?'. Without this there is no answer. Cost data is sensitive — it reveals what systems you run and how big you are.

WHY IT MATTERS

Use `enabled_metric`, not `metric`. The old name is deprecated and disappears in the next major provider version.

WHAT TO SET ON `AZURERM_MONITOR_DIAGNOSTIC_SETTING`

- `target_resource_id` — the account ID with `"/blobServices/default"` appended. The blob *service*, not the account
- `log_analytics_workspace_id` — the workspace from 1.2
- Three log categories: `StorageRead`, `StorageWrite`, `StorageDelete`
- One metric category: `Transaction`

HINT

Search `azurerm_monitor_diagnostic_setting`. Log categories are repeated blocks, one per category.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_monitor_diagnostic_setting" "storage_blob" {
  name                       = "diag-blob"
  target_resource_id        = "${azurerm_storage_account.exports.id}/blobServices/default"
  log_analytics_workspace_id = azurerm_log_analytics_workspace.main.id

  enabled_log {
    category = "StorageRead"
  }

  enabled_log {
    category = "StorageWrite"
  }

  enabled_log {
    category = "StorageDelete"
  }

  enabled_metric {
    category = "Transaction"
  }
}
```

Alerts

Open `terraform/monitoring.tf`.

2.1 Action group

WRITE IT IN `terraform/monitoring.tf` REPLACING `# TODO 2.1`

The delivery list. Alerts don't send themselves — they hand off to an action group, which emails, texts, or calls.

REAL LIFE

When you need to add your manager to every alert, you change one action group instead of editing every rule and forgetting two.

WHY IT MATTERS

`var.alert_emails` is a *list*. A single hardcoded block only handles one address — you need Terraform's `dynamic` blocks, which generate one block per list item.

WHAT TO SET ON `AZURERM_MONITOR_ACTION_GROUP`

- `name`, `resource_group_name`, and `short_name` — `"costalert"` (max 12 chars; Azure uses it as an SMS prefix)
- One email receiver per address in `var.alert_emails`, each with the common alert schema enabled

HINT

Search `azurerm_monitor_action_group`, then look up [dynamic blocks](#). Inside a dynamic block named `email_receiver`, the iterator is also called `email_receiver`, with `.key` and `.value`.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_monitor_action_group" "cost" {
  name                = "ag-${var.name_prefix}"
  resource_group_name = azurerm_resource_group.main.name
  short_name          = "costalert"

  dynamic "email_receiver" {
    for_each = var.alert_emails

    content {
      name                = "email-${email_receiver.key}"
      email_address       = email_receiver.value
      use_common_alert_schema = true
    }
  }

  tags = var.tags
}
```

2.2 Cost anomaly alert

WRITE IT IN `terraform/monitoring.tf` REPLACING `# TODO 2.2`

Azure learns your normal daily spend and tells you when today doesn't match.

REAL LIFE

A monthly budget is slow. If something starts costing \$200/day on the 3rd, the budget stays silent until the running total crosses a threshold — maybe two weeks and \$2,800 later. The anomaly alert catches the shape changing the next day.

WHAT TO SET ON `AZURERM_COST_ANOMALY_ALERT`

- `name`, `display_name`, `subscription_id`
- `email_subject`, `email_addresses`, `message`

HINT

Search "cost anomaly" at the registry. This one emails directly rather than going through an action group.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_cost_anomaly_alert" "main" {
  name           = "anomaly-${var.name_prefix}"
  display_name   = "Daily cost anomaly"
  subscription_id = data.azurerm_subscription.current.id
  email_subject   = "Azure cost anomaly detected"
  email_addresses = var.alert_emails
  message        = "Daily spend deviated from the learned pattern. Check Cost Analysis for the responsi
}
```

P5

The budget

Open `terraform/budget.tf`. This is the centrepiece — get the threshold design right and the rest is plumbing.

3.1 Subscription budget

WRITE IT IN terraform/budget.tf REPLACING # TODO 3.1

A spending limit that emails you as you approach it.

WHY IT MATTERS

Each notification needs `enabled`, `threshold`, `operator`, `threshold_type`, `contact_emails`, and `contact_groups` pointing at the action group from 2.1.

WHAT TO SET ON `AZURERM_CONSUMPTION_BUDGET_SUBSCRIPTION`

- `name`, `subscription_id` — the *full resource ID*, not a bare GUID
- `amount` from `var.monthly_budget`, `time_grain` monthly
- A `time_period` block using the two date variables
- **Four `notification` blocks** — see the table below

THRESHOLD	THRESHOLD_TYPE	WHAT IT MEANS
50	Actual	Half the budget is gone
80	Actual	Getting close — go look
100	Actual	Budget is gone
100	Forecasted	On pace to blow it

This table is the whole lab. The first three are history — they tell you money is already spent. The fourth is a warning that arrives while you can still act. When an interviewer asks what you'd do differently from a default setup, this is the answer.

HINT

Search `azurerm_consumption_budget_subscription`. Notifications are repeated blocks — four of them. The operator is the same for all four.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_consumption_budget_subscription" "main" {
  name            = "budget-${var.name_prefix}"
  subscription_id = data.azure_subscription.current.id

  amount      = var.monthly_budget
  time_grain  = "Monthly"

  time_period {
    start_date = var.budget_start_date
    end_date   = var.budget_end_date
  }

  notification {
    enabled      = true
    threshold    = 50.0
    operator     = "GreaterThanOrEqualTo"
    threshold_type = "Actual"
    contact_emails = var.alert_emails
    contact_groups = [azurerm_monitor_action_group.cost.id]
  }

  notification {
    enabled      = true
    threshold    = 80.0
    operator     = "GreaterThanOrEqualTo"
    threshold_type = "Actual"
    contact_emails = var.alert_emails
    contact_groups = [azurerm_monitor_action_group.cost.id]
  }

  notification {
    enabled      = true
    threshold    = 100.0
    operator     = "GreaterThanOrEqualTo"
    threshold_type = "Actual"
    contact_emails = var.alert_emails
    contact_groups = [azurerm_monitor_action_group.cost.id]
  }

  notification {
    enabled      = true
    threshold    = 100.0
    operator     = "GreaterThanOrEqualTo"
    threshold_type = "Forecasted"
    contact_emails = var.alert_emails
    contact_groups = [azurerm_monitor_action_group.cost.id]
  }
}
```

Export and outputs

4.1 Daily cost export

WRITE IT IN `terraform/export.tf` REPLACING `# TODO 4.1`

Azure writes a CSV of your real usage into storage every day.

REAL LIFE

The portal is fine for 'what am I spending now'. It's useless for 'what did this look like in March' once the view resets. The export is your own permanent copy.

WHY IT MATTERS

Terraform normally works out order by watching which resources reference each other. This export never mentions the role assignment, so Terraform doesn't know it must wait for permissions — and would happily create an export that silently fails its first run. You have to say it out loud.

WHAT TO SET ON `AZURERM_SUBSCRIPTION_COST_MANAGEMENT_EXPORT`

- `name`, `subscription_id`, daily recurrence, and the two period dates
- An `export_data_storage_location` block pointing at the container from 1.6, with `root_folder_path = "daily"`
- `export_data_options` with `type = "ActualCost"` and `time_frame = "MonthToDate"`
- `depends_on` the role assignment from 1.4

HINT

Search `azurerm_subscription_cost_management_export`. Two nested blocks: where it writes, and what it writes. `ActualCost` is what was billed; `AmortizedCost` spreads reservation purchases over time.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
resource "azurerm_subscription_cost_management_export" "daily" {
  name                = "export-${var.name_prefix}-daily"
  subscription_id     = data.azurerm_subscription.current.id
  recurrence_type     = "Daily"
  recurrence_period_start_date = var.budget_start_date
  recurrence_period_end_date   = var.budget_end_date

  export_data_storage_location {
    container_id = azurerm_storage_container.exports.id
    root_folder_path = "daily"
  }

  export_data_options {
    type = "ActualCost"

    time_frame = "MonthToDate"
  }

  depends_on = [azurerm_role_assignment.cost_reader]
}
```

5.1 Outputs

WRITE IT IN `terraform/outputs.tf` REPLACING `# TODO 5.1`

Print the things you'd otherwise go hunting in the portal for.

WHY IT MATTERS

Terraform doesn't require descriptions. Anyone reading your code in six months does.

WHAT TO SET

- Resource group name, storage account name, budget name, the identity's client ID
- Each needs a `description`

HINT

An output block is `output "name" { value = ... description = "..." }`. Reference resources the same way you did inside them.

REFERENCE SOLUTION – TRY IT YOURSELF FIRST

```
output "resource_group" {
  description = "Resource group holding everything in this project."
  value      = azurerm_resource_group.main.name
}

output "storage_account" {
  description = "Storage account receiving the daily cost export."
  value      = azurerm_storage_account.exports.name
}

output "workbook_url" {
  description = "Direct link to the cost dashboard in the Azure portal."
  value      = "https://portal.azure.com/#@/resource${azurerm_application_insights_workbook.cost.id}/wc
}

output "budget_name" {
  description = "Name of the subscription budget."
  value      = azurerm_consumption_budget_subscription.main.name
}

output "cost_reader_client_id" {
  description = "Client ID of the least-privilege identity used to read cost data."
  value      = azurerm_user_assigned_identity.cost_reader.client_id
}

output "monthly_budget_usd" {
  description = "Budget the alert thresholds are calculated against."
  value      = var.monthly_budget
}
```

5.c Final checkpoint

EVERYTHING SHOULD BE GREEN

```
cd ~/Portfolio/projects/01-cost-visibility && ./scripts/check.sh
```

Zero TODOs, validation passing, and every requirement check `[ok]`.

Deploy

Plan first, always. In a real job you plan, someone reviews it, then you apply. That habit is what prevents outages.

7.1 Plan

Shows exactly what will happen without doing it.

```
cd ~/Portfolio/projects/01-cost-visibility/terraform
terraform plan -out=tfplan
```

SCREENSHOT

terraform-plan — Terminal showing Plan: 14 to add, 0 to change, 0 to destroy.

```
./scripts/capture.sh terraform-plan "Plan output: 14 resources to add"
```

7.2 Apply

```
terraform apply tfplan
```

THEN CAPTURE THE API STATE

```
cd .. && ./scripts/snapshot.sh post-apply
```

If it errors, paste it to me exactly. Errors are the most valuable content in this lab — they're what makes the write-up real instead of a tutorial.

SCREENSHOT

terraform-apply — Terminal showing Apply complete! Resources: 14 added and the outputs block.

```
./scripts/capture.sh terraform-apply "Apply complete: 14 resources created"
```

Verify every control

Terraform saying 'created' is not proof a control is on. This is the difference between 'I configured it' and 'I verified it'.

8.1 Portal verification sweep

Five screenshots, one per control, each showing the setting actually enabled.

```
./scripts/snapshot.sh controls-verified
```

SCREENSHOT

resource-group — Resource groups → `rg-costvis`. Show all resources and the tags.

```
./scripts/capture.sh resource-group "All resources deployed with project tags"
```

SCREENSHOT

budget-thresholds — Cost Management → Budgets → `budget-costvis`. **Must show all four conditions including Forecasted.**

```
./scripts/capture.sh budget-thresholds "Budget with three actual thresholds and a forecast alert"
```

SCREENSHOT

storage-security — Storage account → Configuration. Secure transfer **Enabled**, minimum TLS 1.2.

```
./scripts/capture.sh storage-security "Storage enforcing HTTPS-only and TLS 1.2"
```

SCREENSHOT

storage-dataprotection — Storage account → Data protection. Versioning on, soft delete 7 days.

```
./scripts/capture.sh storage-dataprotection "Blob versioning and 7-day soft delete enabled"
```

SCREENSHOT

least-privilege — Managed identity → Azure role assignments. **Exactly one role.**

```
./scripts/capture.sh least-privilege "Identity holds only Cost Management Reader"
```

Prove the control works

The most important phase, and the one almost nobody does.

9.1 Test that least privilege denies

Sign in as the managed identity and try something outside its role. It must fail. Then read cost data, which must succeed.

REAL LIFE

This is what a security audit actually asks for. Not 'is MFA enabled' but 'show me a login being blocked'. Evidence beats configuration.

Tell me when you reach this step and I'll give you the exact commands. They depend on values from your apply outputs.

SCREENSHOT

rbac-denied — Terminal showing `AuthorizationFailed` when the identity attempts a write.

```
./scripts/capture.sh rbac-denied "Identity denied when attempting an action outside its role"
```

SCREENSHOT

rbac-allowed — Terminal showing the same identity successfully reading cost data.

```
./scripts/capture.sh rbac-allowed "Same identity permitted to read cost data"
```

Prove the alert reaches a human

Budgets evaluate every 12–24 hours, so waiting for a real trigger isn't practical. Test the delivery path directly.

10.1 Test the action group

Portal → Action group `ag-costvis` → **Test action group** → Budget alert → Test.

WHY IT MATTERS

The inbox screenshot closes the loop: code → cloud → a human being told. That's the entire point of the system.

SCREENSHOT

actiongroup-test — The test result showing success.

```
./scripts/capture.sh actiongroup-test "Action group test succeeded"
```

SCREENSHOT

alert-email — Your actual inbox with the alert email in it.

```
./scripts/capture.sh alert-email "Alert email received – full path from Terraform to inbox"
```

The dashboard

Terraform built the workbook shell. The cost charts get added in the portal, because hand-writing the query JSON produces dashboards that silently show nothing.

11.1 Add the cost tiles

Workbook → Edit → Add → Add query → Data source: **Azure Resource Manager**.

Then paste the JSON to me and I'll fold it into `workbook.tf`, so the finished dashboard is reproducible from code instead of hand-built.

SCREENSHOT

workbook-query — The ARM query configuration in edit mode.

```
./scripts/capture.sh workbook-query "Workbook ARM data source querying Cost Management"
```

SCREENSHOT

workbook-rendered — The chart actually rendering data.

```
./scripts/capture.sh workbook-rendered "Cost dashboard rendering live data"
```

SCREENSHOT

workbook-editor — Advanced Editor showing the JSON.

```
./scripts/capture.sh workbook-editor "Workbook JSON for export back into Terraform"
```

Next day — real data

Cost data lags 8–24 hours. Come back tomorrow.

12.1 Confirm the export landed

```
./scripts/snapshot.sh export-landed
```

SCREENSHOT

export-blob — Storage → Containers → `cost-exports` → `daily` → the CSV Azure wrote.

```
./scripts/capture.sh export-blob "Daily cost export CSV written to blob storage"
```

SCREENSHOT

cost-analysis-populated — Cost analysis now showing `BUDGET: $25` and a forecast — the direct counterpart to your baseline shot.

```
./scripts/capture.sh cost-analysis-populated "Cost analysis with budget and forecast active"
```

Tear it down

Most portfolios skip this. Doing it demonstrates the exact discipline the project is about.

13.1 Destroy and record the total

```
cd ~/Portfolio/projects/01-cost-visibility/terraform
terraform destroy
```

```
cd .. && ./scripts/snapshot.sh post-destroy
```

This is your most quotable artifact. 'I built, deployed, tested, and documented a cost governance system for \$0.43' is a sentence that lands in an interview.

SCREENSHOT

terraform-destroy — Terminal showing `Destroy complete! Resources: 14 destroyed.`

```
./scripts/capture.sh terraform-destroy "All 14 resources destroyed"
```

SCREENSHOT

final-cost — Cost analysis showing total spend for the whole lab.

```
./scripts/capture.sh final-cost "Total cost of the entire lab"
```